Obligatory assigment

This report explains some of the design choices and answers to theory questions. We do not, however, explain each line of code, as some of the implementations are self explainable when looking at the code and comments found in the project. Please look at the relevant files listed under each task. Under you can see how we split the work between us. We used code reviews to look at each other's work.

Adam, studnr 386100. Task 3,4,7

Jakob, studnr: 385497. Task 1, 2, 5, 6

Task 1

Development explanation

Under is an overview of the SQL statements to create the primary keys

- ALTER TABLE lego_set ADD PRIMARY KEY (id);
- ALTER TABLE lego_brick ADD PRIMARY KEY (brick_type_id, color_id);
- ALTER TABLE lego_inventory ADD PRIMARY KEY (set_id, brick_type_id, color_id);
- ALTER TABLE lego_inventory ADD FOREIGN KEY (set_id) REFERENCES
- lego_set(id);
- ALTER TABLE lego_inventory ADD FOREIGN KEY (brick_type_id, color_id)
- REFERENCES lego_brick(brick_type_id, color_id);

Design choices

Lego_set is given the primary key id because each set has its own individual identifier. Lego brick is given the composite primary key (brick_type_id, color_id) because each brick is identified by its type and color. We chose to have the bricktype first in the key to match with lego_inventory.

Lego_invnetory has the composite primary key (set_id, brick_type_id, color_id). We chose to have set_id first to match with the most common statement being fetch all brick in given set.

The primary key is responsible for each inventory_row to point to a valid set and a valid brick.

Theory question

What SQL statements will go faster?

The primarkey in lego_invnetory ((set_id, brick_type_id, color_id) will make sql statements that use either set_id, brick_type, or color_id combined or individually faster. This is because the database will know there these rows are located instead of having to manually search through all rows (Eldhuset, 2026, Lecture 8.12)

Sql statements that use only brick_type_id or color_id will not be faster because the database is sorted by set_id. If we attempt to only use brick_type or color_id we have to go through the entire database, since the database is not sorted for these columns. (Eldhuset, 2026, Lecture 8.13)

Task 2

Development explanation

SQL-statements to create indexes:

- CREATE INDEX idx_inventory_brick_type ON lego_inventory (brick_type_id);
- CREATE INDEX idx_inventory_color ON lego_inventory (color_id);

Theory Questions:

Why did we create two indexes?

The primary key in lego_inventory is (set_id, brick_type_id, color_id). Since we created this primary key with “set_id” first, we will need to create indexes for the two other columns. One for “brick_type_id” and one for “color_id”. This will make statements that only use “brick_type_id” or “color_id” faster because they are utilizing the index. This was an issue mentioned in Task 1.

Why does this give better performance?

Without the indexes the database have to go through the all of Lego_inventory. With the indexes the database looks up the given information, finds the location of where it is stored, and then looks up the location. This means that we no longer need to look through the entire lego_inventory to find what we are looking for! (Eldhuset, 2026, Lecture 8.2)

Example statements:

SELECT set_id FROM lego_inventory WHERE brick_type_id = '29bc01';

SELECT set_id FROM lego_inventory WHERE color_id = 90;

Timing

Without indexes:

```
lego-db=# SELECT set_id FROM lego_inventory WHERE brick_type_id = '608p03';
 set_id
--------
 306-1
 454-1
 9355-1
(3 rows)

Time: 203.450 ms
```

```
lego-db=# SELECT set_id FROM lego_inventory WHERE color_id = 90;
Time: 130.370 ms
```

With indexes:

```
lego-db=# SELECT set_id FROM lego_inventory WHERE brick_type_id = '608p03';
 set_id
--------
 306-1
 454-1
 9355-1
(3 rows)

Time: 0.858 ms
```

```
lego-db=# SELECT set_id FROM lego_inventory WHERE color_id = 90;
Time: 12.345 ms
```

As shown by the timing examples, the sql statements that utilize the indexes are much more efficient!

Task 3

Relevant files:

- Server.py

Theory question

Identifying the problem

```
rows = ""

start_time = perf_counter()
conn = psycopg.connect(**DB_CONFIG)
try:
    with conn.cursor() as cur:
        cur.execute("select id, name from lego_set order by id")
        for row in cur.fetchall():
            html_safe_id = html.escape(row[0])
            html_safe_name = html.escape(row[1])        Aasmund Eldhuset, 2 weeks ago • Project framework.
            existing_rows = rows
            rows = existing_rows + f'<tr><td><a href="/set?id={html_safe_id}">{html_safe_id}</a></td><td>{html_safe_name}</td></tr>\n'
    print(f"Time to render all sets: {perf_counter() - start_time}")
finally:
    conn.close()
```

The program uses string concanitation while going through all rows. Since strings are immutable in python this means that for each row we will copy the entire string and then add the new row. This results in the phenomenom Painters algorithm. The algorithm will therefore have complexity of Θ (n^2). (Eldhuset, 2026, Lecture 2)

Development explanation

For improving the performance, we will need to remove the painters algorithm phenomenom. In our solution, we chose to create a list and append each row. When we

have iterared through each row, we will use “join” to create a string based on the list. This will have a complexity of (n k)) (Eldhuset, 2026, Lecture 2)

Result before implementation

```
adamt@DESKTOP-TU4UEAQ MINGW64 ~/Documents/MCP project/dave3606-project-2026 (main)
$ python server.py
 * Serving Flask app 'server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 109-591-198
127.0.0.1 - - [24/Mar/2026 15:23:45] "GET / HTTP/1.1" 200 -
Time to render all sets: 6.619930599932559
127.0.0.1 - - [24/Mar/2026 15:23:47] "GET /sets HTTP/1.1" 200 -
Time to render all sets: 6.658390900003724
127.0.0.1 - - [24/Mar/2026 15:23:58] "GET /sets HTTP/1.1" 200 -
```

Without any changes to the original code given for this assignment it took around 6,5 seconds to load all rows. This is shown in the image below.

Result after implementation

```
adamt@DESKTOP-TU4UEAQ MINGW64 ~/Documents/MCP project/dave3606-project-2026 (main)
$ python server.py
 * Serving Flask app 'server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 109-591-198
127.0.0.1 - - [24/Mar/2026 15:35:35] "GET / HTTP/1.1" 200 -
Time to render all sets: 0.07303610001690686
127.0.0.1 - - [24/Mar/2026 15:35:40] "GET /sets HTTP/1.1" 200 -
```

After the implementation it now takes around 0.07 seconds to load all rows. Much more efficient.

Task 4

Relevant files:

- Server.py
- Templates/sets.html

Theory question

File handle leaks:

There were several file handle leaks in the project code. The file handle leaks where in the methods index, sets, and legoset. The issue is that the methods are not closing the files. We fixed all the leaks with changing how we open the files to use with open, that automatically close the files. This can be seen below.

```
@app.route("/")
def index():
    #Automatically close the file when done
    with open("templates/index.html") as f:
        template = f.read()

    return Response(template)

#Helper method for checking if the given encoding is
```

Compress response:

The responses are compressed before being returned using gzip.compress(). This can be seen in the image below.

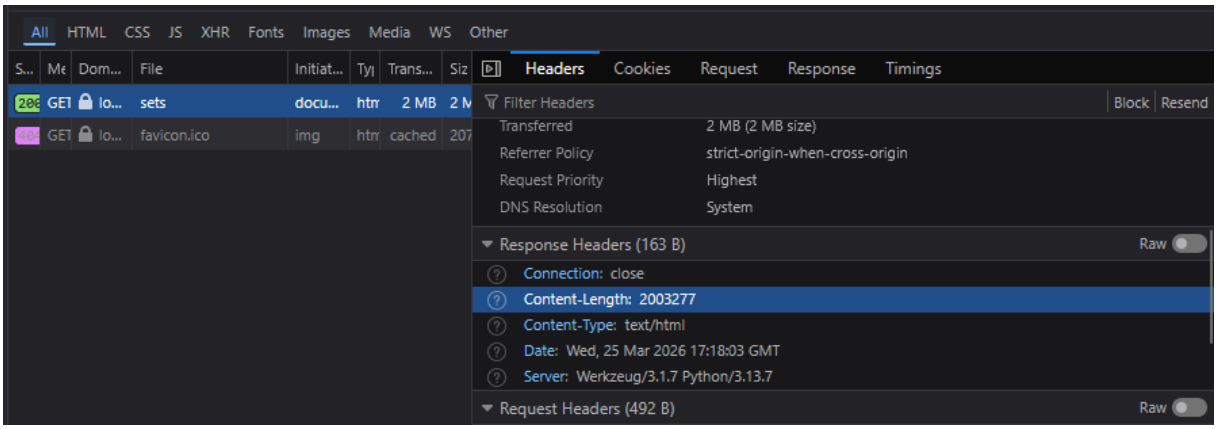
```
#Compress page with GZIP
compressed_page = gzip.compress(encoded_page)
        You, 19 hours ago • Task 7 …

return Response(compressed_page, content_type="text/html; charset=" + encoding_type
```

Sets endpoint

The endpoint retrieves the encoding and then checks if it is valid. The endpoint uses UTF-8 as default encoding, which is enforced if given encoding is illegal or missing. The HTML will be encoded with given encoding type and sent with the response. We replaced the <meta charset="UTF-8"> with a placeholder as hinted by the task. To see our implementation please look at the relevant files.

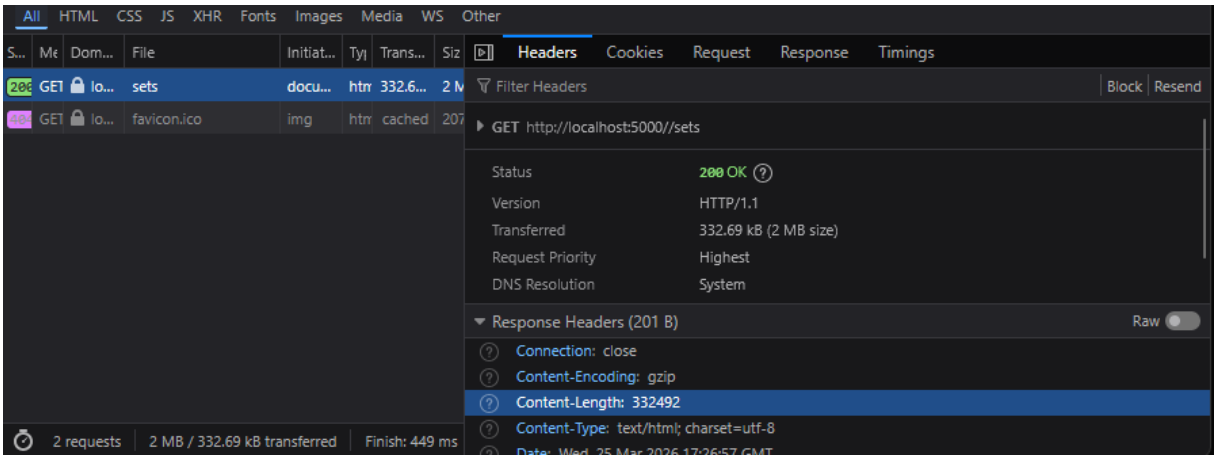
Inspect Content-length and see what effect encoding has on response size:

Content length before task implementation:



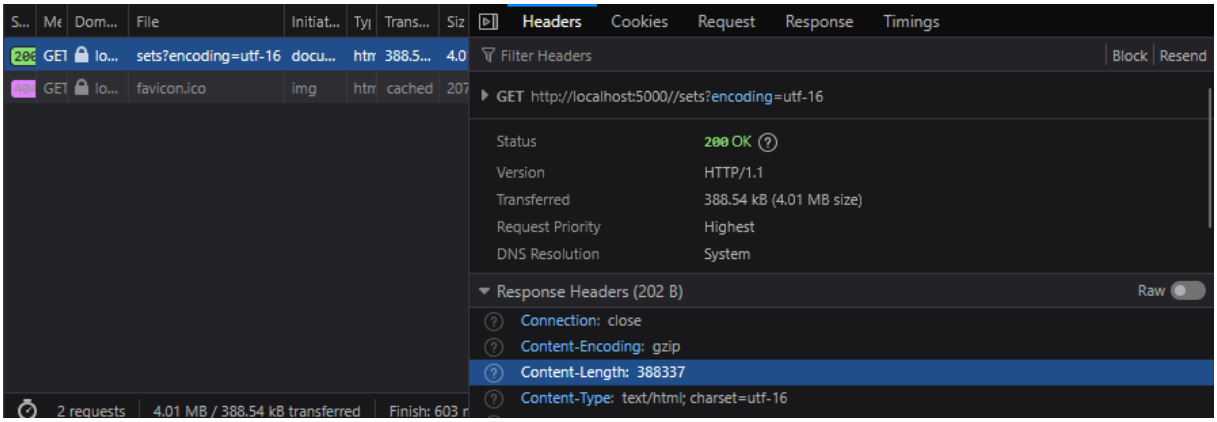
As seen in the picture above the content length is 2003277

Content length with UTF-8



As seen in the picture above the content length is 332492. Noticeable smaller than the content length before implementation.

Content length with UTF-16



As seen in the picture above the content length is 388337. Noticeable smaller than the content length before implementation, but bigger than the content length with UTF-8. This is because UTF-16 use more bytes for certain characters. (Eldhuset, 2026, Lecture 7)

Task 5

Relevant files:

- Server.py
- Read_binary_set.py

JSON

The JSON structure we created looks like this:


```
{
    "id": "00-2",
    "name": "Weetabix Promotional House 1",
    "year": 1976,
    "category": "Catalog: Sets: LEGOLAND: Building",
    "bricks": [
        {
            "name": "Black Arch 1 x 4",
            "color_id": 11,
            "count": 2
        },
        {
            "name": "Black Brick 1 x 1",
            "color_id": 11,
            "count": 6
        },
        {
            "name": "Black Brick 1 x 2",
            "color_id": 11,
            "count": 8
        },
```


We decided to structure the JSON similar to the lego_set structure to keep it simple. This is why we are saving the id, name, year, and category. We are also saving the bricks from the given set in a list with given brick name, color_id and count.

Binary file format

We chose to use Big endian for our binary format because we found it more intuitive. The data fields have +n because lengths are variable. Our binay file format can be found in the table below.

**In this table n means length of string in bytes

Data field	Data filed size	Description of field	Design choices
Magic bytes	4 bytes	Lego, identifies the fileformat	Chosen to identify fileformat
Set_Id	2+n bytes	Two byte big endian + an UTF-8 String	
Set_name	2+n bytes	Two byte big endian + an UTF-8 String	
Year	2 bytes	Big endian unsigned short. Is 0 if unknown.	Is unsiged because year cant be negative
Category	2+n byte	Two bytes big endian + an UTF-8 String	
Brick_count	4 bytes	Big endian unsigned int	Is unsiged because count can't be negative
Per kloss, Name	2+n bytes	Two bytes big endian + an UTF-8 String	
Per kloss Color_id	4 bytes	Big endian signed int	
Per kloss, Count	4 bytes	Big endian signed int	

The console application:

The console application was created to read and print our binary file format. The application will first read the 2 byte length prefix, read corresponding number of bytes and then decode. The console application can be found in the relevant files with relevant comments.

Task 6

Relevant files:

- Server.py
- Templates/set.html

Development explanation

We created the cache on the serverside to use least recently used (LRU). The cache can store 100 entries. If 100 are stored, the entry that was used the longest time ago will be replaced. This is the concept of LRU. The entries are stored in an orderedDict. With a cache hit, we will use the method move_to_end to move the entry to the end. This is because the entry was just used and therefore have to be moved back to ensure LRU principle. (Eldhuset, 2026, Lecture 9)

The operations will have a O(1) complexity because OrderedDict is built as a hash table and a double linked list. (Eldhuset, 2026, Lecture 9) The endpoint does also have a cache control header that lets the browser save a copy and use it for requests in the next 60 seconds. With this the browser does not have to collect from the server within the time limit.

Timing:

```
Cache MISS for 10179-1: 0.0439s
127.0.0.1 - - [27/Mar/2026 15:34:01] "GET /api/set?id=10179-1 HTTP/1.1" 200 -
Cache HIT for 10179-1: 0.0001s
127.0.0.1 - - [27/Mar/2026 15:34:01] "GET /api/set?id=10179-1 HTTP/1.1" 200 -
```

Here we can see the efficiency with a cache miss and a cache hit. As the image suggests, the cache hit makes the request faster.

Task 7

Relevant files:

- Server.py
- Database.py
- Test_database.py

Development explanation

Refactor endpoints

We refactored all endpoints to use dependency injection instead of handling the database individually. The database logic was made into separate methods instead of in the endpoints. These methods were called with the database as a parameter from the endpoints.

Database class

First we created the database class that can be seen in the image below. It has the requested method execute_and_fetch_all. It takes the query as a parameter and calls the cursor with given query. The class does have a close function that closes both the

connection and the cursor.

```
#Data base class
You, 22 hours ago | 1 author (You)
class Database:
    def __init__(self):
        #Connection
        self.connect = psycopg.connect(**DB_CONFIG)
        #Cursor created with connection
        self.cursor = self.connect.cursor()

    #Method for fetching all object with given query
    def execute_and_fetch_all(self, query, vars=None):
        #Perform query
        self.cursor.execute(query, vars)
        #Save result and return
        query_result = self.cursor.fetchall()
        return query_result


    #Method for closing connection
    def close(self):
        #Close the connection and cursor
        self.cursor.close()
        self.connect.close()
```

Testing

We created tests for the get_sets and api_set endpoints. The tests follow the same principles being creating mock data, call function and then checking the result. We created the mock to prevent the tests from running on the actual database. The mock database class can be seen below.

```
You, 5 hours ago | 1 author (You)
class MockDatabase:
    def __init__(self, set_data, bricks_data=None):
        self.set_data = set_data
        self.bricks_data = bricks_data or []

    def execute_and_fetch_all(self, query, vars=None):
        #Save query to mock for future testing
        self.endpoint_query = query
        if "lego_set" in query:
            return self.set_data
        return self.bricks_data
```

Reference list:

Eldhuset, Å. (2026). Lecture 8 [PDF]. OsloMet.

Eldhuset, Å. (2026). Lecture 2 [PDF]. OsloMet.

Eldhuset, Å. (2026). Lecture 7 [PDF]. OsloMet.

Eldhuset, Å. (2026). Lecture 9 [PDF]. OsloMet.